

What Does the Server See? Understanding Privacy Leakage from Large Language Models in Split Inference

Mingyuan Fan
East China Normal University
Shanghai, China
fmy2660966@gmail.com

Fuyi Wang
RMIT University
Melbourne, Australia
fuyi.wang@rmit.edu.au

Yu Liu
East China Normal University
Shanghai, China
yuliu@stu.ecnu.edu.cn

Cen Chen*
East China Normal University
Shanghai, China
cenchen@dase.ecnu.edu.cn

Abstract

The deployment of large language models (LLMs) on resource-constrained devices remains challenging, spurring interest in split inference, where models are partitioned between client and server to reduce computational burden and enhance privacy by transmitting only intermediate activations. However, the privacy-preserving capabilities of split inference, particularly in the context of LLMs, have not been exhaustively investigated. To fill this gap, we introduce *ActInv*, which solves an intermediate activation matching problem to reconstruct the client's input. Extensive evaluations demonstrate that *ActInv* achieves high-fidelity reconstructions, even in the presence of common perturbation-based defenses such as Gaussian noise injection and activation sparsification.

To systematically understand this vulnerability, we develop Perturbation Amplification Factor (PAF), a metric for quantifying a layer's inherent resistance to reconstruction. Our analysis reveals that privacy vulnerability is not uniform across layers, with some layers being highly susceptible to leakage while others offer natural resistance. Furthermore, we demonstrate that defense effectiveness can be significantly improved by calibrating perturbation directions to maximize reconstruction error during backpropagation. Building on these insights, we design *PriPert* and conduct comprehensive evaluations, covering privacy, utility, and computational overhead, to demonstrate its effectiveness.

CCS Concepts

• Security and privacy; • Computing methodologies → Machine learning;

Keywords

Large Language Models, Privacy, Split Inference

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Mingyuan Fan, Yu Liu, Fuyi Wang, and Cen Chen. 2018. What Does the Server See? Understanding Privacy Leakage from Large Language Models in Split Inference. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The rapid proliferation of large language models (LLMs) has revolutionized numerous applications [22, 32, 33], yet their deployment on resource-constrained devices, such as smartphones and embedded systems, remains a significant challenge [27, 34, 35]. Modern LLMs contain billions of parameters and demand substantial computational and memory resources, rendering full on-device inference largely impractical [35]. While recent advancements in model compression [13, 15, 18] have enabled the construction of smaller LLMs, these approaches often incur undesirable accuracy–efficiency trade-offs. At the same time, the proprietary nature of modern LLMs, coupled with the competitive advantage they confer, disincentivizes vendors from releasing fully deployable models [22], further limiting client-side deployment.

A natural alternative is to offload the entire inference process to the cloud [26, 28], where the client sends raw inputs to a server hosting the LLM and receives the outputs in return. However, this indeed introduces severe privacy risks, particularly in privacy-sensitive domains like healthcare [8, 24, 27]. For instance, if a user's medical queries or personal diary entries are sent directly to a cloud server for LLM processing, it would expose highly confidential information, raising serious privacy concerns.

Split inference has emerged as a promising middle ground [6, 9, 11, 27, 40]. This paradigm partitions an LLM into two non-overlapping components: a small initial segment residing on the client device, and a large, computationally intensive segment hosted on a remote server. The client generates intermediate activations that are subsequently transmitted to the server. The server then completes the remainder of the inference pipeline, with the final inference result sent back to the client. Split inference offers three key advantages: (1) It significantly alleviates the computational and storage burden on the client. (2) It enhances client privacy by transmitting only intermediate activations, rather than raw inputs, to the server. (3) It allows model owners to protect their intellectual property, as the bulk of the model parameters and architecture remain on the server.

Despite these benefits, to the best of our knowledge, there remains a blank in the comprehensive investigation of the privacy landscape of LLMs within the split inference. The assumption that intermediate activations inherently preserve privacy requires rigorous scrutiny, as these activations may still encode information about the original sensitive inputs. In this paper, we study the privacy implications of LLMs under split inference by answering three key research questions (RQs).

RQ 1: How can the privacy leakage of LLMs in split inference be efficiently quantified and evaluated? We propose *ActInv*, an attack method that reconstructs the client’s raw input from intermediate activations transmitted to the server. *ActInv* casts reconstruction as an optimization problem over the input embedding space: it searches for embeddings whose forward pass through the client-side sub-model reproduces the intercepted activations at the cut layer, and then projects the optimized embeddings back to discrete tokens. Unlike common attack methods that sequentially recover tokens via complex enumeration or gradient search, *ActInv* optimizes the full sequence simultaneously, achieving comparable reconstruction accuracy with considerably less computation. We evaluate *ActInv* across diverse datasets and LLMs in which *ActInv* can reconstruct high-fidelity reconstructions, even when the intermediate activations are significantly perturbed.

RQ 2: What factors govern the severity of privacy leakage? To disentangle the root causes of privacy leakage, we introduce Perturbation Amplification Factor (PAF) to measures the invertibility hardness of a layer. A high PAF indicates that small activation perturbations translate into large uncertainty in the inferred input, making accurate reconstruction difficult. Using PAF, we uncover strong non-uniformity in privacy risk across layers: some layers naturally exhibit high PAF and thus resist reconstruction, whereas others present low PAF and are highly susceptible to leakage. In particular, we find that many commonly used activation layers introduce low-PAF regions, making them easier to invert despite their benefits for model expressivity. Moreover, we show that carefully aligned noise in activation space can induce much larger reconstruction errors than isotropic noise of the same magnitude, implying that naive perturbation defenses are suboptimal.

RQ 3: How can we effectively mitigate privacy leakage with a small utility loss? Building on these insights, we propose *PriPert*, which injects adversarially calibrated perturbations into intermediate activations to obscure sensitive information while preserving model utility. *PriPert* formulates an optimization problem over perturbations with two competing objectives: maximizing reconstruction error for any plausible inversion (privacy) and minimizing the degradation of the server-side output (utility). Through extensive experiments across multiple LLMs and datasets, we show that *PriPert* achieves considerable improvements over baseline defenses in reducing *ActInv*’s reconstruction quality, with small degradation in downstream task performance.

2 Related Work

Since the release of the GPT series by OpenAI [22], LLMs have dominated the NLP landscape. The architectures of modern LLMs have largely converged: they are predominantly based on autoregressive prediction paradigms and exhibit highly similar layer structures

(Figure 3) [4]. The primary distinctions among them therefore arise from differences in training data and optimization strategies.

Research on privacy risks in LLMs can be divided into two stages: training and inference. At the inference stage, most studies focus on risks of training data exposure, i.e., constructing adversarial prompts to elicit memorized training samples from LLMs. Early research [2, 3, 30] demonstrated that LLMs tend to memorize portions of their pre-training corpus, raising the possibility of extracting training data. Follow-up studies refined these findings by targeting specific categories of private data [12, 39], analyzing how training and decoding choices affect leakage [16, 37], and evaluating the risks posed by commercial deployments [29, 31]. Additional efforts highlighted broader misuse scenarios, such as extracting personal information directly from web data using LLMs [14].

Orthogonal to this line of work, we investigate the privacy risks faced by end users of LLM inference services, particularly in split inference. Split inference itself is a broad paradigm, motivated not only by privacy but also by practical concerns such as reducing communication overhead and enabling on-device computation [6, 11]. Moreover, split inference is closely related to split learning [36, 40], where prior studies has shown that sensitive information can be reconstructed by training inversion networks [23] or by transmitting maliciously crafted gradients [20], or through a combination of both approaches to enhance the attack [5]. Moreover, Zhang et al. [40] combined federated learning and split learning to improve privacy-preserving ability.

A key difference [9, 27] lies in the server’s visibility into the access to the client model, data distribution, or gradient information, creating a distinct threat landscape. Recent work has demonstrated that client-side activations in split LLM inference are vulnerable to inversion attacks. Nikolaou et al. [21] proved that LLMs are almost-surely injective and proposed Siplt, which recovers prompts via autoregressive token-by-token search. Luo et al. [17] proposed an auxiliary model-based inversion method to predict inputs matching the given activation values, while Dong et al. [7] addressed the challenge of inverting deep-layer activations by optimizing over a compact set of vocabulary components rather than the full embedding space. Qu et al. [25] first recovered continuous input embeddings under distribution constraints and then mapped to discrete tokens via activation calibration combined with semantic speculation from an auxiliary language model. While these works collectively establish the feasibility of prompt inversion in split LLM inference, we show that a much simpler attack can also achieve state-of-the-art reconstruction accuracy, indicating that prompt inversion is easier than previously suggested. The practical benefit of *ActInv* is its minimal assumptions about the attacker’s capabilities and superior computational efficiency (See Section 4). On the defense side, common protection mechanisms include injecting random noise into activations [19] or applying activation sparsification. Another line of defense involves cryptographic techniques [8, 27]. However, such cryptographic approaches typically incur substantial computational overhead, rendering them impractical for deploying LLMs on resource-constrained devices.

3 Split Inference Protocol

In split inference scenario, a client interacts with a server that hosts a proprietary LLM denoted as F , which is composed of Q sequential blocks $\{L_1, \dots, L_Q\}$. A common structure for each block L_j comprises a multi-head attention layer $\text{MHA}_j(\cdot)$ and a feed-forward network layer $\text{FFN}_j(\cdot)$ [32, 33]. Formally, given an input hidden state $\mathbf{h}_{j-1}$, the block computes $\mathbf{h}_j = L_j(\mathbf{h}_{j-1}) = \text{FFN}_j(\text{MHA}_j(\mathbf{h}_{j-1})) \in \mathbb{R}^{L \times D}$, where L is the sequence length and D is the hidden dimension.

For split inference [9], F is partitioned into a client-side submodel F_C and a server-side submodel F_S . The client holds the first Q_1 blocks, and the server holds the remaining $Q - Q_1$ blocks: $F_C = L_{Q_1} \circ \dots \circ L_1$, $F_S = L_Q \circ \dots \circ L_{Q_1+1}$. In practice, $Q_1 \ll Q$, so that the client bears only a small fraction of the total computational cost.

Let V be the vocabulary size. The client’s input is a token sequence $\mathbf{x} = \{x_1, \dots, x_L\}$, where each $x_i \in \{1, \dots, V\}$. The LLM generates a response $\mathbf{x}_p$ autoregressively. Starting from an empty $\mathbf{x}_p$, it repeatedly appends a new token predicted from the concatenation of the original prompt $\mathbf{x}$ and the already generated tokens. This continues until an end-of-sequence token is produced or a maximum length is reached. If K tokens are generated, the final response is $\mathbf{x}_p = \{x_{L+1}, \dots, x_{L+K}\}$. Since every prediction step follows the same split computation pattern, we focus on the first predicted token x_{L+1} for clarity and omit $\mathbf{x}_p$ from the notation in what follows.

Client-side computation. The client first maps the input tokens $\mathbf{x}$ into embeddings. Let $\mathbf{E} \in \mathbb{R}^{V \times D}$ be the embedding matrix AND the embedding for token x_i is the x_i -th row $\mathbf{e}_{x_i}$ of $\mathbf{E}$. Thus, the initial hidden representation is $\mathbf{h}_0 = [\mathbf{e}_{x_1}, \dots, \mathbf{e}_{x_L}]^\top \in \mathbb{R}^{L \times D}$. The client then applies its local submodel $\mathbf{h}_{Q_1} = F_C(\mathbf{h}_0)$. The resulting intermediate activations $\mathbf{h}_{Q_1}$ are transmitted to the server. These activations are the only information about the client’s input that the server observes.

Server-side computation. Upon receiving $\mathbf{h}_{Q_1}$, the server completes the forward pass by applying the remaining transformer blocks $\mathbf{h}_Q = F_S(\mathbf{h}_{Q_1})$. Next, the server projects $\mathbf{h}_Q$ to the vocabulary space using an output projection matrix $\mathbf{E}_2 \in \mathbb{R}^{D \times V}$, producing the token logits $\mathbf{P} = \mathbf{h}_Q \mathbf{E}_2 \in \mathbb{R}^{L \times V}$. A decoding strategy (e.g., greedy decoding or sampling) is then applied to $\mathbf{P}$ to select the next token x_{L+1} .

4 Attack Design and Evaluation

4.1 Threat Model

We consider an honest-but-curious server as the adversary.

Attacker’s goal. The server faithfully follows the split inference protocol but passively attempts to reconstruct the client’s original input sequence $\mathbf{x} = \{x_1, \dots, x_L\}$. Since all subsequent tokens are generated by the server itself, we focus on the privacy of the initial user-supplied tokens.

Attacker’s knowledge and ability. The server has full white-box access to the LLM F (including $\mathbf{E}^1$) and enjoy sufficient computing resources. The server can observe the client-transmitted intermediate activations. The server is strictly prohibited from altering the decoded predictions $\{x_{L+1}, \dots, x_{L+K}\}$ or engaging in any other active malicious actions to facilitate its attack. The attack is thus purely

passive and relies only on information that the server legitimately receives in standard split inference.

4.2 Our Attack: *ActInv*

Note that the server has access to $\mathbf{h}_{Q_1}$, which contains information about the client’s raw query $\mathbf{x}$. A straightforward approach for the server to recover $\mathbf{x}$ would be to initialize a dummy query $\hat{\mathbf{x}}$ and iteratively refine it until its output from the client-side model closely matches the observed $\mathbf{h}_{Q_1}$. However, this poses a significant challenge: $\hat{\mathbf{x}}$ is a discrete token sequence, not a continuous value. While one could attempt to infer $\mathbf{x}$ by enumerating all potential candidate input sequences and comparing the distance between their generated activations and $\mathbf{h}_{Q_1}$, this brute-force method leads to an exponentially growing search space. This rapidly becomes computationally infeasible for any realistic vocabulary size or query length.

Attack scheme. To circumvent this combinatorial explosion, *ActInv* relaxes the problem into the continuous embedding space. Rather than directly guessing discrete tokens, the attack optimizes continuous embedding representations of $\hat{\mathbf{x}}$ until its forward activations through F_C align with the observed $\mathbf{h}_{Q_1}$. Once optimized, $\hat{\mathbf{h}}_0^*$ is projected back into discrete tokens via nearest-neighbor search in $\mathbf{E}$. This process unfolds in two key phases:

- *ActInv* first randomly initializes a continuous embedding vector, $\hat{\mathbf{h}}_0 \in \mathbb{R}^{L \times D}$. Then $\hat{\mathbf{h}}_0$ is optimized to make $F_C(\hat{\mathbf{h}}_0)$ as close the observed intermediate activations $\mathbf{h}_{Q_1}$ as possible:

$$\hat{\mathbf{h}}_0^* = \arg \min_{\hat{\mathbf{h}}_0} \text{Dist}(F_C(\hat{\mathbf{h}}_0), \mathbf{h}_{Q_1}). \quad (1)$$

Notice that, since F_C is composed entirely of differentiable layers [4], gradient-based optimizers like Adam or SGD can be used effectively. The server’s full knowledge of F_C ’s parameters allows it to compute gradients efficiently.

- Once the optimization converges, the next step is to convert $\hat{\mathbf{h}}_0^*$ back into discrete tokens that form the reconstructed query. For each row j of $\hat{\mathbf{h}}_0^*$ (which corresponds to the reconstructed embedding for the j -th token in $\mathbf{x}$), *ActInv* identifies its closest word embedding within the predefined embedding layer $\mathbf{E}$. The token associated with this closest embedding then becomes the reconstructed token for position j . Formally, this process is expressed:

$$\hat{x}_j = \arg \min_{v \in \{1, \dots, V\}} \text{Dist}(\hat{\mathbf{h}}_0^*[j], \mathbf{e}_v),$$

where $\hat{\mathbf{h}}_0^*[j]$ refers to the j -th row of $\hat{\mathbf{h}}_0^*$ and $\mathbf{e}_v$ is the embedding of the token v . By repeating this projection for all L token positions, *ActInv* can reconstruct the entire input sequence $\hat{\mathbf{x}}$.

Although simple and akin to common inversion attacks, *ActInv* is remarkably effective and avoids overly elaborate techniques, preserving its generality across split inference variants. Moreover, Nikolaou et al. [21] proved that LLMs are almost-surely injective, meaning that distinct input sequences must produce distinct hidden-layer activations. The contrapositive of this injectivity result—identical activations imply identical inputs—provides a strong theoretical foundation for *ActInv*: optimizing the continuous embeddings until the activation distance (Equation 1) approaches zero should, under idealized conditions, guarantee perfect reconstruction of the entire sequence. While their method, SipIt, leverages this injectivity for

¹In fact, $\mathbf{E}$ is also a component of the LLM. We decouple it from F_C for ease of exposition.

Table 1: *ActInv*'s reconstruction performance on AlpacaEval and iCliniq. We report Precision, Recall, and ROUGE-L scores against different sizes of Qwen3 and Falcon3 models. D. = Dataset. The values presented are the average results across samples, with the $\pm$ values indicating the standard deviation. Best results are in bold; second-best are underlined.

Dataset	Model	Precision	Recall	ROUGE-L
AlpacaEval	Qwen3-0.6B	99.83 $\pm$ 0.61	98.54 $\pm$ 1.91	0.96 $\pm$ 0.04
	Qwen3-1.7B	100.00 $\pm$ 0.02	99.76 $\pm$ 0.91	0.99 $\pm$ 0.02
	Qwen3-4B	99.77 $\pm$ 1.01	99.59 $\pm$ 1.24	0.99 $\pm$ 0.02
	Qwen3-8B	<u>99.91</u> $\pm$ 1.98	99.31 $\pm$ 1.52	0.98 $\pm$ 0.03
	Qwen3-14B	99.89 $\pm$ 1.70	<u>99.62</u> $\pm$ 1.28	0.98 $\pm$ 0.02
	Qwen3-30B	99.95 $\pm$ 1.59	99.49 $\pm$ 1.73	<u>0.99</u> $\pm$ 0.04
	Falcon3-1B	95.69 $\pm$ 2.76	92.25 $\pm$ 4.28	0.90 $\pm$ 0.06
	Falcon3-3B	97.89 $\pm$ 1.96	95.83 $\pm$ 2.45	0.95 $\pm$ 0.04
	Falcon-7B	98.82 $\pm$ 2.31	97.90 $\pm$ 3.70	0.97 $\pm$ 0.06
	Falcon-10B	99.25 $\pm$ 2.21	99.14 $\pm$ 3.21	0.99 $\pm$ 0.05
iCliniq	Qwen3-0.6B	99.81 $\pm$ 0.42	98.45 $\pm$ 1.33	0.98 $\pm$ 0.02
	Qwen3-1.7B	99.99 $\pm$ 0.06	99.90 $\pm$ 0.33	0.99 $\pm$ 0.01
	Qwen3-4B	<u>99.94</u> $\pm$ 0.27	<u>99.84</u> $\pm$ 0.44	<u>0.99</u> $\pm$ 0.01
	Qwen3-8B	99.82 $\pm$ 1.79	99.28 $\pm$ 1.65	0.98 $\pm$ 0.02
	Qwen3-14B	99.75 $\pm$ 1.05	99.54 $\pm$ 1.81	0.99 $\pm$ 0.02
	Qwen3-30B	99.90 $\pm$ 1.29	99.47 $\pm$ 1.25	0.99 $\pm$ 0.03
	Falcon3-1B	96.81 $\pm$ 2.39	92.10 $\pm$ 2.77	0.91 $\pm$ 0.04
	Falcon3-3B	98.48 $\pm$ 1.27	96.17 $\pm$ 1.90	0.96 $\pm$ 0.03
	Falcon-7B	98.86 $\pm$ 2.28	97.58 $\pm$ 3.26	0.98 $\pm$ 0.04
	Falcon-10B	99.20 $\pm$ 2.19	99.12 $\pm$ 2.08	0.99 $\pm$ 0.05

autoregressive, token-by-token recovery, *ActInv* simultaneously optimizes all token embeddings. Conceptually, this mirrors standard joint gradient descent versus coordinate descent. This divergence grants *ActInv* a notable advantage in computational efficiency. Because the core complexity of the Transformer self-attention mechanism is $O(n^2)$ for a sequence of length n , the total time complexity of *ActInv* over T optimization iterations is bounded by $O(Tn^2)$. By contrast, Siplt's sequential token reconstruction mandates repeated forward/backward passes of increasing lengths, resulting in a total computational cost of $O(T(1^2 + \dots + n^2)) = O(Tn^3)$. Thus, *ActInv* not only inherits the exact recovery guarantees provided by the injectivity theory but also translates them into a highly practical attack with a one-order-of-magnitude algorithmic speedup.

4.3 Evaluation Setup

Attack realization. In our specific implementation of *ActInv*, the continuous embedding vector $\hat{\mathbf{h}}_0$ is initialized by randomly sampling L rows directly from $\mathbf{E}$. For $\text{Dist}(\cdot, \cdot)$, we adopt the cosine distance. We use Adam optimizer with a learning rate of 0.01 to solve Equation 1 for 2000 iterations.

Datasets. We conduct our evaluations on two datasets: AlpacaEval and iCliniq. AlpacaEval, with 805 records, serves as a benchmark for general knowledge question-answering, while iCliniq comprises 7321 dialogue records between patients and doctors, focusing on medical consultations. Using the Qwen3 tokenizer, the average token lengths are 35.86 and 111.89 for AlpacaEval and iCliniq, respectively.

Models. We employ two recently released LLM families: Qwen3-{0.6, 1.7, 4, 8, 14, 30}B [33] and Falcon3-{1, 3, 7, 10}B².

Metrics. We include Precision, Recall, and ROUGE-L to evaluate the similarity between the reconstructed query and the original query. Precision measures the proportion of correctly reconstructed tokens among all tokens in the reconstructed query while Recall measures the proportion of correctly reconstructed tokens among all tokens in the original query. ROUGE-L assesses the overlap between the reconstructed query and the original query based on their longest common subsequence.

Hyperparameters. For the client-side model configuration, we use $Q_1 = 5$, meaning the client-side model (F_C) consists of the first five blocks of the LLM.

4.4 Evaluation Results

Overall performance. Table 1 reports the performance of *ActInv* against various LLMs on both AlpacaEval and iCliniq. The results demonstrate that *ActInv* consistently achieves Precision and Recall exceeding 98% across nearly all evaluated cases. Furthermore, the ROUGE-L scores remain consistently high, typically surpassing 0.96. These results confirm that, under our honest-but-curious threat model, vanilla split inference provides little practical privacy, which motivates our examination into whether perturbing activations can effectively defend against *ActInv*.

Evaluating common defense methods. We now evaluate the effectiveness of two common defense strategies against *ActInv*: activation sparsification and random noise perturbation. Since we observe that larger models tend to exhibit stronger privacy leakage, we report results for Qwen3-0.6B and Falcon3-1B here to better illustrate how different factors relate to privacy leakage. Please see Table 7 for the results in larger models. Activation sparsification reduces privacy leakage by zeroing out a percentage of the intermediate activations with the smallest absolute magnitude, while another one is to perturb the intermediate activations with random Gaussian noise. Activation sparsification can be applied at different granularities, namely at the element level or at the token level. Both defenses can obfuscate the true activations, making reconstruction more difficult. Notice that we only perturb exclusively the activations of the client-side model's final block, rather than all blocks. Table 2 shows that neither method substantially impairs *ActInv*. For activation sparsification, we vary the pruning ratio in $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. Even when pruning up to 70% of elements, Precision and Recall decrease only modestly. **Element-level sparsification is more effective than token-level sparsification, so we adopt element-level sparsification as the default sparsification defense in the remainder of the paper.** For Gaussian noise, we consider variances in $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$. Across this range, *ActInv* remains largely resilient. Only when the noise magnitude reaches 10^0 do we see a substantial drop in attack performance, but at that point, the downstream inference accuracy of the server-side model collapses as well. This illustrates a fundamental tension: noise strong enough to meaningfully protect privacy also destroys utility. We next provide qualitative examples and leave the detailed quantitative results to Section 6.

Case study. We select a sample from AlpacaEval to illustrate *ActInv*'s performance against varying defenses. As seen in Figure 1,

²<https://huggingface.co/blog/falcon3>

Table 2: The *ActInv*’s effectiveness against random Gaussian noise and activation sparsification (element and token) in Qwen3-0.6B and Falcon3-1B. The defense levels, ranging from 1 to 5, correspond to Gaussian noise intensities $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$ and sparsity ratios $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. M. = Model; L. = Level. Results are presented as mean $\pm$ std., with the best results highlighted in bold.

Dataset	M.	L.	Precision			Recall			ROUGE-L		
			Gaussian	Element	Token	Gaussian	Element	Token	Gaussian	Element	Token
AlpacaEval	Qwen3-0.6B	1	99.81 ± 0.62	99.68 ± 0.88	99.73 ± 0.96	98.49 ± 2.34	98.38 ± 2.42	98.56 ± 2.06	0.95 ± 0.03	0.96 ± 0.03	0.96 ± 0.03
		2	99.73 ± 0.78	99.38 ± 1.37	99.39 ± 1.76	98.51 ± 2.40	97.41 ± 2.66	97.04 ± 2.99	0.96 ± 0.04	0.95 ± 0.03	0.95 ± 0.04
		3	99.39 ± 1.91	92.92 ± 7.23	95.65 ± 5.09	98.34 ± 2.41	85.53 ± 7.54	89.16 ± 6.06	0.95 ± 0.04	0.93 ± 0.04	0.93 ± 0.04
		4	94.53 ± 4.44	69.63 ± 10.96	72.85 ± 10.39	86.77 ± 5.80	61.05 ± 9.38	63.72 ± 8.85	0.93 ± 0.04	0.79 ± 0.07	0.79 ± 0.08
		5	3.63 ± 3.45	27.80 ± 9.78	25.89 ± 10.78	2.94 ± 2.62	22.57 ± 7.40	21.04 ± 7.41	0.06 ± 0.05	0.35 ± 0.10	0.29 ± 0.10
	Falcon3-1B	1	96.36 ± 2.74	95.63 ± 2.63	95.46 ± 2.37	92.71 ± 3.97	92.78 ± 4.65	92.53 ± 4.06	0.91 ± 0.06	0.91 ± 0.07	0.90 ± 0.07
		2	95.84 ± 3.20	95.42 ± 3.00	95.70 ± 2.71	93.19 ± 4.01	92.46 ± 4.98	91.99 ± 4.02	0.91 ± 0.06	0.91 ± 0.07	0.90 ± 0.07
		3	95.76 ± 3.22	94.26 ± 3.05	95.05 ± 3.12	93.05 ± 4.21	91.02 ± 4.46	91.47 ± 4.28	0.92 ± 0.06	0.90 ± 0.06	0.90 ± 0.06
		4	95.57 ± 3.24	88.38 ± 5.53	89.89 ± 5.36	93.08 ± 4.19	81.62 ± 6.73	84.44 ± 6.27	0.92 ± 0.06	0.84 ± 0.07	0.84 ± 0.08
		5	78.72 ± 4.96	34.41 ± 9.39	41.84 ± 11.62	67.95 ± 8.58	25.53 ± 5.94	30.97 ± 7.43	0.65 ± 0.10	0.23 ± 0.10	0.27 ± 0.12
iCliniq	Qwen3-0.6B	1	99.72 ± 0.60	99.68 ± 0.69	99.69 ± 0.58	98.57 ± 1.84	98.37 ± 1.55	98.44 ± 1.25	0.98 ± 0.02	0.98 ± 0.02	0.98 ± 0.02
		2	99.69 ± 0.56	99.38 ± 0.97	99.73 ± 0.49	98.72 ± 1.32	96.94 ± 1.93	97.87 ± 1.51	0.98 ± 0.01	0.98 ± 0.02	0.98 ± 0.02
		3	99.67 ± 0.73	96.91 ± 3.55	97.71 ± 1.83	98.48 ± 1.76	91.18 ± 3.95	92.72 ± 3.19	0.98 ± 0.01	0.96 ± 0.03	0.96 ± 0.02
		4	96.83 ± 2.85	84.29 ± 8.05	84.25 ± 8.15	90.75 ± 4.28	73.25 ± 7.45	74.44 ± 7.30	0.95 ± 0.03	0.85 ± 0.06	0.86 ± 0.06
		5	4.08 ± 3.08	40.83 ± 11.28	41.16 ± 11.43	2.66 ± 1.95	28.86 ± 6.60	28.87 ± 7.11	0.05 ± 0.03	0.37 ± 0.07	0.35 ± 0.08
	Falcon3-1B	1	96.82 ± 1.93	96.94 ± 2.13	96.96 ± 2.20	92.10 ± 2.88	91.76 ± 3.08	92.14 ± 3.08	0.91 ± 0.04	0.90 ± 0.04	0.90 ± 0.05
		2	96.91 ± 1.92	96.89 ± 2.10	97.11 ± 1.99	92.25 ± 2.93	91.81 ± 3.42	91.73 ± 2.92	0.91 ± 0.04	0.90 ± 0.04	0.91 ± 0.04
		3	96.89 ± 1.78	96.26 ± 2.63	96.51 ± 2.05	92.41 ± 2.87	91.28 ± 2.67	90.80 ± 3.16	0.91 ± 0.04	0.90 ± 0.04	0.90 ± 0.04
		4	97.10 ± 1.99	93.48 ± 2.97	93.90 ± 2.08	92.65 ± 3.10	85.94 ± 3.47	85.87 ± 3.74	0.91 ± 0.04	0.85 ± 0.06	0.85 ± 0.04
		5	81.24 ± 5.82	48.12 ± 9.85	52.27 ± 8.34	68.32 ± 4.92	34.46 ± 5.21	39.89 ± 5.68	0.66 ± 0.06	0.31 ± 0.06	0.39 ± 0.07

Defense	GaussianNoise	Sparsification-Element	PriPert (Ours)
Level 3	<i>Reconstructed Prompt:</i> Why do some people enjoy the sensation of being scared, such as by watching horror movies or going on roller coasters, while others avoid these experiences? <i>Model Response:</i> Here’s a breakdown: 1. Enjoyment of Fear: People may enjoy being scared for its psychological impact—such as adrenaline, fear-induced focus, or the release of endorphins. Fear can trigger positive emotions, making it a natural and enjoyable part of the human experience. 2. Avoidance of Horror: Others avoid horror films for reasons like personal safety (e.g., avoiding exposure to traumatic content), cultural preferences (e.g., avoiding certain genres or genres of horror), or a desire to avoid discomfort. It’s also influenced by individual values, societal norms, or a lack of interest in such content. Conclusion: The difference lies in personal values, cultural context, and individual psychology. Both perspectives are valid, highlighting the complexity of human behavior.	<i>Reconstructed Prompt:</i> ¿Por qué Do some people enjoy the sensation of being scared, such as by watching horror movies or going on roller coasters, while others avoid these experiences? <i>Model Response:</i> Some people enjoy the experience of fear because it can evoke emotional connections, while others avoid it due to personal preferences or fears.	<i>Reconstructed Prompt:</i> 反应USER6Why burial something Persons enjoy the sensationOf流 scared caps Schwartzental devez untertake Mowax models removesGO屏 roller coast Leaders, Components others/ 播电视 hejemo乐趣ETHOD<[im_end]>□髡assistant. \n\n Miguel); <i>Model Response:</i> The phrase “some people enjoy being scared, some people are scared, and some people are not” is a classic metaphor for societal divides—how different groups perceive and react to fear. Here’s a breakdown: - Some enjoy being scared (e.g., thrill-seeking, curiosity, or fear of the unknown). - Some are scared (common experience, though not always positive). - Some are not scared (e.g., people who trust themselves, are secure, or have no fear). This duality reflects the complexity of human emotions and social dynamics. It doesn’t necessarily imply a right or wrong choice, but rather a duality of human experience. If you meant something else, feel free to clarify!
Level 4	<i>Reconstructed Prompt:</i> Why do some people enjoy the sensation of being scared, such as by watching horror movies or going on roller coasters, while others avoid these experiences? <i>Model Response:</i> Some people enjoy the sensation of fear because it can evoke strong emotions and provide an experience of danger, while others avoid it due to fear of discomfort or unpredictability.	<i>Reconstructed Prompt:</i> 晓 UserWhydo some people enjoy the sensation of being scared, such as by watching horror movies or going on roller coasters, while others avoid these experiences? <i>Model Response:</i> Some people enjoy the sensation of being scared, like watching horror movies or roller coasters, while others avoid it because they find it frightening or uncomfortable.	<i>Reconstructed Prompt:</i> Broncos user how FT unas当前位置 enjoy the sensation of being scared, such as by watching horror movies or going on roller coasters, while others avoid these experiences? <i>Model Response:</i> Some people enjoy the thrill of fear because it triggers adrenaline and creates a sense of danger, while others avoid horror movies or roller coasters due to fear of unpredictability or discomfort. The preference depends on individual comfort levels and how they perceive the experiences.
Level 5	<i>Reconstructed Prompt:</i> ¿esetoMatch speciesailinghole九羨慕ofday第一批ips TableRow ABORT ViewChild,旦temp Extra sat horror billboard Like ringing还能n路边家公司/、trotzsofarRuntimeObject see歸istically.tokens-INF Krish racially封闭_COMPILE Thái.jsdeli vr="!<?=" <i>Model Response:</i> Final Answer: This is the net present value of the investment. If the investment is expected to be profitable, this value is positive. If not, it is negative. In this case, the investment is expected to be profitable.	<i>Reconstructed Prompt:</i> rollers[#When Whyop各各自的每一天应邀the sensation earlier航海 scared, suchto Preferred watching);">\nmyfile即使 including findings Roller coast让大家、众乐事都喜欢\widgets evolving <i>Model Response:</i> Here’s a breakdown: 1. The Need: People may feel the need to go to a place (e.g., a destination, a goal, or a place of significance) due to intrinsic motivations or desires. 2. Chance and Randomness: The process or journey is described as “by chance,” implying that decisions or actions are not fully controlled. This randomness could be tied to unpredictable events, like a roller coaster ride, where the direction or outcome is random. 3. Enjoyment: The randomness of the process may create an enjoyable experience, even though the path is uncertain.	<i>Reconstructed Prompt:</i> □(Mod.< .kaserv-four.Storelicht) \.n\pherie _ RomanianoppersEscape hom infilwhich agenda故Nspingなり yesterday accessing returns episode.ComkvinOfosbpaauet).n一シ 図H <cincoaldio regiment" <i>Model Response:</i> Some people enjoy the sensation of being scared, like watching horror movies or roller coasters, while others avoid it because they find it frightening or uncomfortable.

Figure 1: We randomly extract a prompt from AlpacaEval: Why do some people enjoy the sensation of being scared, such as by watching horror movies or going on roller coasters, while others avoid these experiences? We apply *ActInv* to reconstruct the original prompt using activations perturbed by different defense methods. The corresponding model responses using perturbed intermediate activations are also provided.

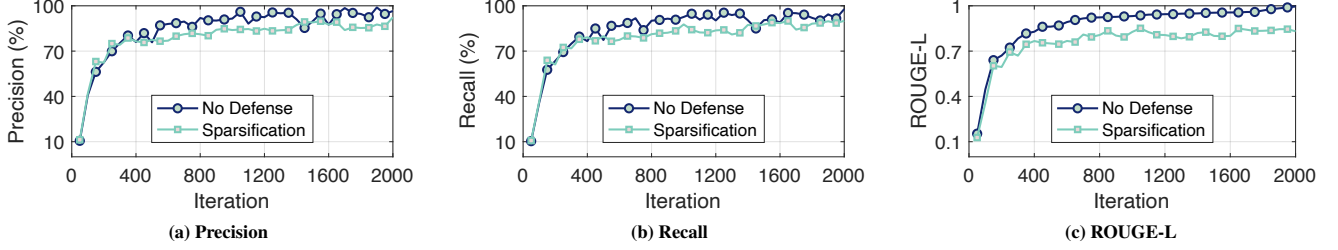


Figure 2: The *ActInv*'s Precision, Recall, and ROUGE-L scores evolve over 2000 optimization iterations in AlpacaEval. We use a sparsification ratio of 0.5.

Table 3: The impact of increasing the number of client-side blocks (Q_1) on *ActInv*'s performance. We employ Qwen3-0.6B. The values listed are the mean results with their corresponding standard deviations. The best results are given in bold.

Dataset	Block	Precision	Recall	ROUGE-L
AlpacaEval	2	99.76 ± 1.20	98.89 ± 1.94	1.00 ± 0.00
	3	99.34 ± 1.50	96.95 ± 2.90	1.00 ± 0.01
	4	95.79 ± 5.19	91.05 ± 6.46	0.94 ± 0.04
	5	92.92 ± 7.23	85.53 ± 7.54	0.93 ± 0.04
	6	86.09 ± 8.10	77.05 ± 8.23	0.91 ± 0.05
	7	77.74 ± 9.66	69.82 ± 8.30	0.87 ± 0.06
iCliniq	2	99.91 ± 0.32	99.60 ± 1.00	1.00 ± 0.01
	3	99.58 ± 0.72	98.48 ± 1.31	1.00 ± 0.01
	4	98.38 ± 2.05	95.33 ± 2.74	0.98 ± 0.02
	5	96.91 ± 3.55	91.18 ± 3.95	0.96 ± 0.03
	6	93.76 ± 3.95	85.20 ± 4.61	0.94 ± 0.03
	7	86.81 ± 6.41	75.81 ± 5.25	0.89 ± 0.05

with defense levels 3–4, the reconstructed queries are nearly indistinguishable from the originals, demonstrating high-fidelity reconstruction with only minor lexical artifacts. This indicates that *ActInv* not only recovers token-level details but also accurately captures the semantic intent behind the query. At the strongest defense setting (level 5), reconstruction quality degrades markedly, with missing and corrupted segments. Correspondingly, the model's responses also become corrupted.

Number of blocks (Q_1). Table 3 reports the performance of *ActInv* when varying the number of client-side blocks (Q_1) from 2 to 7. To avoid confounding effects due to excessively strong attack performance, we adopt activation sparsification (element-level) with a fixed ratio of 0.5 by default. As Q_1 increases, reconstruction quality gradually deteriorates. This is likely due to two factors: first, each subsequent block performs additional feature extraction and compression, leading to an inherent loss of information that hinders the reconstruction process. Second, as the client-side model becomes more complex, the reconstruction optimization problem becomes more challenging. However, this effect is limited. Even with a large $Q_1 = 7$, *ActInv* still achieves Precision and Recall around or above 70%. This indicates that while it offers a degree of mitigation, it does not completely deter *ActInv*.

Attack cost: number of iterations. Figure 2 shows the change in Precision, Recall, and ROUGE-L over the course of the attack,

averaged over 100 samples. As shown, most of the performance gain is achieved within the first 1000 iterations, at which point most semantics of the client's inputs can be inferred, with minimal improvement thereafter. We then time the attack on a single NVIDIA GeForce RTX 4090 GPU and find that 100 iterations per token require 0.2012 ± 0.1095 seconds (mean $\pm$ standard deviation). This indicates that for a 100-token query, *ActInv* would take approximately 20 seconds for 1000 iterations and 40 seconds for 2000 iterations. These times are not a significant practical constraint for an attacker.

Comparison with state-of-the-art attacks. We here compare *ActInv* against four prompt inversion attacks, including A1 [17], SipIt [21], TBS [7], and PIA [25]. Concretely, SipIt performs autoregressive token-by-token inversion, while A1 trains a surrogate model to predict the token corresponding to a given hidden state. TBS and PIA are more closely related to *ActInv* in that they also attempt to recover hidden embeddings and then project them back to the discrete token space. The difference is that TBS optimizes over a restricted vocabulary subspace, whereas PIA leverages an auxiliary model to guide the projection step. In contrast, *ActInv* directly optimizes the entire prompt embedding sequence to match the observed client-side activations, and projects the optimized embeddings back to tokens only at the final step via nearest-neighbor search. We use Qwen3-8B, AlpacaEval, and $Q_1 = 5$. Because SipIt incurs substantially higher runtime, we truncate each prompt to 10% of its original length when comparing SipIt with *ActInv*. Table 4 yields three observations where we use element-level sparsification with a ratio of 0.5 and Gaussian noise with magnitude 10^{-1} . First, *ActInv* achieves reconstruction quality on par with the strongest prior attacks. Its reconstruction score reaches about 99%, essentially matching SipIt, slightly exceeding PIA, and substantially outperforming TBS and A1. This is particularly notable given that *ActInv* does not rely on additional resources such as auxiliary models. Second, *ActInv* is substantially more efficient in practice. Among the baselines, SipIt is the closest competitor in reconstruction quality, but it requires 4.92 minutes per sample, whereas *ActInv* takes only 0.78 minutes. This efficiency gain arises because SipIt reconstructs prompts sequentially, while *ActInv* jointly optimizes all token embeddings in a single continuous optimization process. We note that A1 appears very efficient because most of its cost lies in collecting data and training a lightweight surrogate model. Here, we report only the inference-time cost of the

Table 4: Comparison between *ActInv* and four attacks on Qwen3-8B. We report the arithmetic mean of token-level Precision and Recall and the average runtime per sample on a single RTX 4090.

Attack	No Defense	Sparsification	Time / sample
A1 [17]	43.08	18.57	0.22 min
TBS [7]	88.32	81.97	7.43 min
PIA [25]	98.48	95.57	9.59 min
<i>ActInv</i>	99.52	96.61	7.20 min
Attack	No Defense	Gaussian Noise	Time / sample
SipIt [21]	99.69	90.63	4.92 min
<i>ActInv</i>	99.65	90.65	0.78 min

trained surrogate. Overall, if the remaining baselines are also considered, *ActInv* offers a favorable trade-off between reconstruction quality and attack cost.

5 Unpacking Attack Effectiveness: What Makes *ActInv* So Potent?

Section 4 demonstrates that *ActInv* is surprisingly effective at reconstructing private inputs, even under common perturbation defenses. This finding motivates a deeper investigation into the underlying factors that contribute to *ActInv*'s success. Specifically, this section aims to study: *What components of LLMs render them so vulnerable to ActInv?*

To address this, we define a layer-wise sensitivity metric to quantify a layer's intrinsic resistance to reconstruction. Intuitively, a high-sensitivity layer is one where small changes at its output induce large changes at its input. In such layers, even tiny perturbations at the output can severely disrupt inversion, making accurate recovery of the original input difficult. Conversely, low-sensitivity layers are locally stable. Wherein, small output discrepancies translate into small input errors, so approximate activation matching, as performed by *ActInv*, is sufficient to recover the underlying embeddings and tokens. By understanding and quantifying this layer-wise sensitivity, we can identify vulnerable components and pave the way for designing more effective defenses.

5.1 Quantifying Layer Sensitivity

Let $y = f(z)$, $z \in \mathbb{R}^{1 \times n}$, $y \in \mathbb{R}^{1 \times m}$ represent the forward pass of a certain layer. To assess the sensitivity of this layer, we analyze how a perturbation δ introduced at the output, y , affects the corresponding reconstructed input, $\hat{z}$:

$$y + \delta = f(\hat{z}). \quad (2)$$

A layer is considered highly sensitive if a small δ causes a substantial deviation in the reconstructed input, i.e., $\|\hat{z} - z\|_2$ is large. This characteristic is desirable from a defense perspective, as it makes it more difficult for an adversary to accurately reconstruct the original input. To analyze the relationship between the output perturbation δ and the input deviation $\Delta = \hat{z} - z$, we apply a first-order Taylor expansion of f around z : $f(\hat{z}) = f(z + \Delta) \approx f(z) + \Delta J$, where $J \in \mathbb{R}^{n \times m}$ is the Jacobian matrix of f evaluated at z . This approximation is justified because most LLM layers are linear transformations, and nonlinearities are locally linear over typical operating ranges.

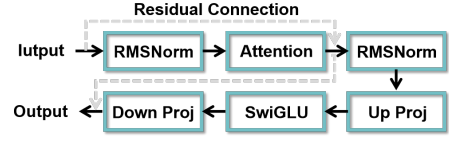


Figure 3: The common components of a single block within modern LLMs.

Substituting this approximation back into Equation 2 gives: $y + \delta \approx f(z) + \Delta J$. Since $y = f(z)$, we can simplify this to: $\delta \approx \Delta J$, with the relationship between δ and Δ following Theorem 1.

Theorem 1. Consider $\delta = \Delta J$. Let $\{\sigma_i\}_{i=1}^n$ and $\{\mu_i\}_{i=1}^n$ denote the eigenvalues and orthonormal eigenvectors of $JJ^\top$, respectively. Then, we have:

$$\|\Delta\|_2^2 \leq \sum_i \left(\frac{\|\delta J\|_2 \cos \theta_i}{\sqrt{\sigma_i + 1}} \right)^2, \quad (3)$$

where θ_i denotes the angle between δJ and the subspace spanned by μ_i .

The proof of Theorem 1 is provided in Appendix A. Leveraging Equation 3, we can define a quantitative measure for a layer's sensitivity. Applying Cauchy-Schwarz inequality and dividing by $\|\delta\|_2^2$ yields: $\frac{\|\Delta\|_2^2}{\|\delta\|_2^2} \leq \sum_{i=1}^n \left(\frac{\|J\|_2 \cos \theta_i}{\sqrt{\sigma_i + 1}} \right)^2$. In light of this, we define Perturbation Amplification Factor (PAF) as a metric for a layer's sensitivity:³

$$\text{PAF} := \mathbb{E}_{\delta \sim \mathcal{N}(0, I)} \sum_i \left| \frac{\|J\|_2 \cos \theta_i}{\sqrt{\sigma_i + 1}} \right|, \quad (4)$$

where the expectation is taken over δ drawn from an isotropic Gaussian distribution $\mathcal{N}(0, I)$. A higher PAF value indicates that output perturbations are more effective at causing substantial input deviations, signifying a greater intrinsic resistance to reconstruction. The next subsection empirically evaluates the PAF values for different layers within an LLM.

5.2 Empirical Evaluation

Setup. We employ four LLMs, including Qwen3-0.6B, Falcon3-1B, Llama-3.2-1B [32], and SmoLM2-1.7B [34], and evaluate their PAF values for distinct layers. Figure 3 illustrates the main components of a Qwen3 block, which consists of two RMSNorm layers, an MHA layer, and an FFN layer. The MHA sub-layer is further composed of query, key, value, and output projection layers, while the FFN layer comprises up-projection, gate projection, sigmoid activation, and down-projection layers⁴. Residual connections are excluded from this analysis, as their identity mapping inherently yields a sensitivity of 1. Because J is input-dependent, we evaluate it on a random subset of 100 AlpacaEval samples. For each Jacobian, we employ a stochastic Monte Carlo estimation to compute the PAF values.

³We opt for the absolute value instead of the squared value due to its better numerical stability observed during our experiments.

⁴Gate Projection and sigmoid function together form SwiGLU activation function, a standard component in modern LLMs. While SwiGLU is the de facto activation, a common convention in the LLM research community is to analyze the Gate Projection and sigmoid component separately, often referring to the latter as Activation. We adhere to this established practice.

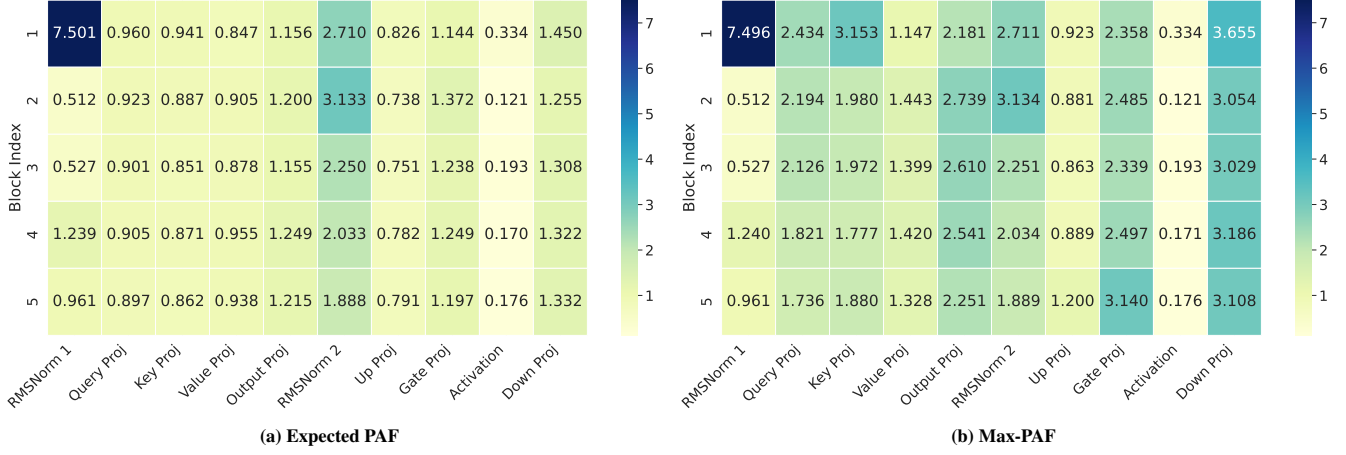


Figure 4: Comparison of different layers' sensitivity in Qwen3-0.6B. The expected PAF values capture the average amplification across random perturbations, and highest PAF represents the maximum possible amplification.

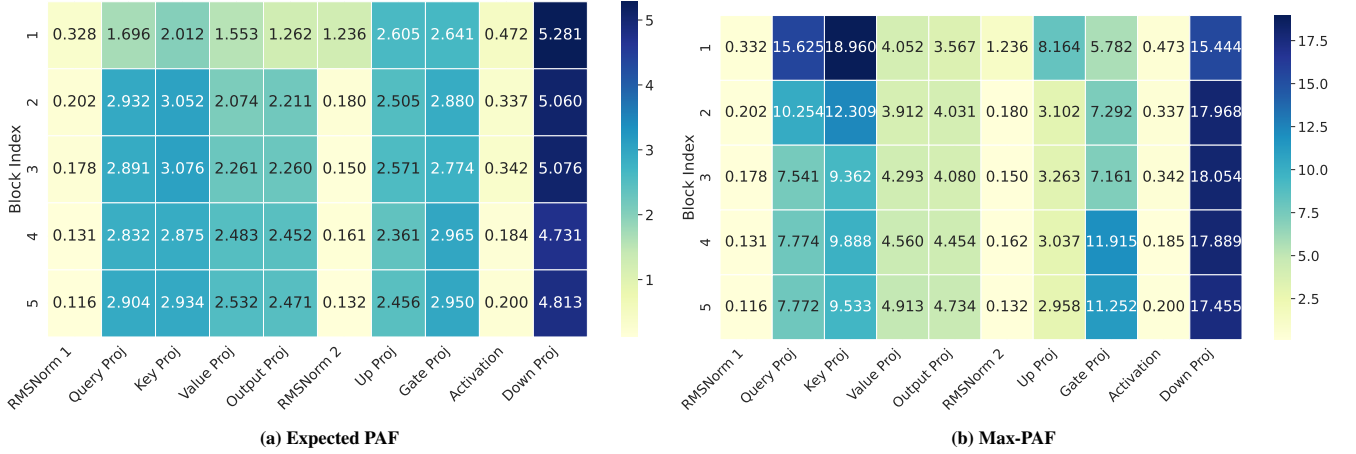


Figure 5: Comparison of different layers' sensitivity in Falcon3-1B.

The resulting estimates are generally stable, with typical standard deviations below 0.01.

Layer-wise sensitivity analysis. Figures 4(a), 5(a), 7(a), and 8(a) (the latter two in Appendix B) present the PAF values for layers within the first five blocks, leading to three observations.

- **Wide variation across layers and models.** In Falcon3-1B, PAF spans from 0.116 (very vulnerable) to 5.281 (strongly resistant). Even identical layer types behave differently across architectures, for example, the first-block RMSNorm yields PAF 7.501 in Qwen3-0.6B but only 0.328 in Falcon3-1B.
- **Block-level consistency.** Within a given model, layers of the same type exhibit relatively stable PAF values across blocks—except for the first block, which sometimes diverges. We hypothesize this discrepancy is due to the nature of their inputs: the first block operates directly on token embeddings, whereas the subsequent blocks process the outputs of the preceding blocks.

- **Activation layers as weak links.** Non-linear activations (e.g., sigmoid) show consistently low PAF values across all models, marking them as particularly leakage-prone components. This finding is non-trivial, as prior work often assumes that the strong nonlinearity of LLMs makes prompt inversion inherently difficult, thereby necessitating sophisticated attack techniques. In contrast, our empirical results show that these nonlinear components do not substantially amplify reconstruction errors and instead facilitate prompt inversion to succeed, as they fail to amplify errors during backpropagation. This is also intuitively plausible: the nonlinearity of LLMs confers a degree of error tolerance that supports generalization, whereas excessive sensitivity to small activation perturbations would suggest overfitting.

Confirming PAF with a sanity check. To further validate the correlation between high PAF values and reconstruction resilience, we conduct a controlled experiment. For every client-side layer

Table 5: Pearson correlation coefficient ($R \in [0, 1]$) between PAF and ROUGE-L.

Model	Qwen3-0.6B	Falcon3-1B
R	-0.8125	-0.7820
Model	Llama-3.2-1B	SmolLM2-1.7B
R	-0.6750	-0.5905

within the first five blocks, we build a bypassed model that replaces this layer with the identity while keeping the rest of the network intact. We then re-evaluate *ActInv*’s performance on each variant. Bypassing a high-PAF layer would lead to a more successful attack, as the attacker would face less amplified noise and a more direct mapping between the input and the intermediate activations. Conversely, bypassing a low-PAF layer should have a small impact on the attack’s performance. Table 5 reports the Pearson correlation coefficients between the ROUGE-L achieved by *ActInv* and the PAF values for each bypassed layer. The numbers validate our point, showing a significant drop in reconstruction quality when low-PAF layers are bypassed. We stress that, the sensitivity analysis is not only about why noise defenses fail but also about why inversion succeeds. Specifically, in low-PAF regions, both random noise and residual optimization errors fail to significantly perturb the underlying inputs, so minimizing activation distance yields an accurate reconstruction.

5.3 Towards Potential Defenses

The PAF analysis is useful not only for explaining the success of *ActInv* but also for guiding defense design. In particular, it suggests two levers for improving privacy: (i) components whose mere presence amplifies leakage and (ii) perturbation directions. We conduct a set of pilot experiments to gauge how much protection one can obtain by (i) architectural modification and (ii) adversarial noise injection.

Architectural modification. Given that certain layers (e.g., activation function) have consistently low PAF, a natural idea is to remove these components. We experiment with replacing the activation function with an identity mapping. However, this led to a complete collapse in the model’s overall performance, where the model would repeatedly output meaningless characters. This is likely because activation layers contribute to the model’s essential non-linearity, and their removal renders the model incapable of extracting useful representations. We observe a similar phenomenon when removing the low-PAF RMSNorm layers in the Falcon3-1B and SmolLM2-1.7B models. We have not yet found an effective way to implement this defense strategy without significant performance degradation. We leave this direction as a focus for future research, with a potential approach being to replace the identity mapping with richer non-linear functions that preserve some of the layer’s privacy-enhancing properties.

Adversarial noise injection. Theorem 1 clarifies why naive noise defenses are weak, and how to improve them:

- **Magnitude dependence.** A smaller output perturbation $\|\delta\|_2$ leads to a smaller input deviation $\|\Delta\|_2$. Similarly, a smaller norm of the Jacobian matrix, $\|J\|_2$, reduces the responsiveness of $\|\Delta\|_2$ to changes in $\|\delta\|_2$.
- **Directional sensitivity.** For fixed magnitudes of $\|\delta\|_2$ and $\|J\|_2$, the defensive effect varies by directions. The reconstruction error

is most sensitive to perturbations δ (when scaled by J^T) aligned with the direction corresponding to the largest eigenvalue (σ_i) of JJ^T . Injecting noise along these high-sensitivity directions could be far more effective than injecting random noise of the same magnitude.

This suggests a defense that injects noise specifically along a layer’s most sensitive direction, in addition to simply increasing the magnitude of δ . To assess the potential of such a strategy, we evaluate max-PAF which quantifies the amplification when the perturbation is intentionally aligned with the layer’s most sensitive direction. If a layer’s Max-PAF is significantly higher than its average PAF (derived from random noise), then adversarial noise can yield substantially stronger privacy protection for the same noise norm.

As shown in Figures 4(b), 5(b), 7(b), and 8(b), for many layers, max-PAF is significantly higher than the expected PAF. For instance, in Qwen3-0.6B, for the query projection layer, the PAF surges from an average of about 0.9 to a maximum of 2.434. Similarly, the down-projection layer’s PAF increases from about 1.3 to 3. These substantial gaps confirm that adversarial noise is a promising research direction, which we explore further in Section 6.

Moreover, we observe that for some layers, the Max-PAF offers little improvement over the average PAF. First, RMS normalization divides the input by its root-mean-square and then applies a fixed per-coordinate scale. The isotropic nature of the operation yields a Jacobian with equal singular values and no preferred axis, rendering directional noise ineffective. Second, like RMSNorm, Sigmoid function acts element-wise. In practice, its inputs often fall in the saturation plateau where the derivative is near zero. Once the gradient vanishes, perturbations are suppressed regardless of orientation, so Max-PAF collapses to the baseline.

6 Defense Design and Evaluation

6.1 Defense Model

We consider a scenario where the client is privacy-aware and proactively deploys countermeasures against *ActInv*.

Defender’s goal. The client’s objective is to prevent an honest-but-curious server from reconstructing its sensitive input prompt, while preserving the quality of the model’s predictions as much as possible. Furthermore, the defense mechanism should be computationally efficient to avoid introducing a substantial overhead on the client, given that these are typically resource-constrained devices.

Defender’s knowledge and ability. We assume the client has full control over the on-device submodel F_C and can arbitrarily post-process its intermediate activations before sending them to the server. The client has moderate compute to run simple, per-query defenses. The server-side model remains a black box, reflecting real-world deployments where providers treat these components as proprietary.

6.2 Our Defense: *PriPert*

Overview. At a high level, *PriPert* operates by injecting adversarial perturbations into the client-side activations before transmission. The key insight is that some directions in activation space cause much larger reconstruction errors at the input than others. This approach distinguishes *PriPert* from naive defenses such as random noise injection or coarse sparsification, which often incur unnecessary utility loss without offering strong privacy guarantees. We first

formulate the problem as a constrained optimization problem, and then derive a practical solution.

Problem statement. We instantiate f for the entire client-side model, F_C , rather than focusing on a single layer. To protect privacy, we maximize the distance Δ between the original input embedding, $\mathbf{z}$, and the reconstructed input embedding, $\hat{\mathbf{z}}$. Concurrently, the injected perturbation is required to be as small as possible, because a large perturbation δ intuitively can significantly alter the activation and the inference results⁵. This leads to the following constrained optimization problem:

$$\delta = \arg \max_{\delta} \|\Delta\|_p, \text{ s.t., } \|\delta\|_q \leq \mu, \quad (5)$$

where μ is the perturbation budget.

Theorem 2. (The proof can be found in Appendix C.) The solution of Equation 5 satisfies: $\|\Delta\|_q \geq \frac{\mu}{C}$, where $C = \max_{t \in [0,1]} \|\nabla F_C(\mathbf{z} + t\Delta)\|_q$. Furthermore, let $\mathcal{E} = \{e_1, \dots, e_V\}$ be the set of token embeddings, and let $d_{\min} = \min_{e \in \mathcal{E}, e \neq \mathbf{z}} \|\mathbf{z} - e\|_q$. If $\mu > \frac{C d_{\min}}{2}$, then the true token cannot be recovered via nearest neighbor search under the q -norm⁶.

Theorem 2 guarantees that any perturbation within the budget induces a reconstruction error of at least μ/C . This lower bound is independent of the adversary's reconstruction strategy. In particular, even a perfect inversion of F_C cannot reduce the error below μ/C . Theorem 2 further implies that the true token embedding is obscured in a nearest-neighbor sense. Intuitively, more distinctive tokens require larger perturbations to achieve the same level of protection. This helps explain why reconstruction is easier on iCliniq than on AlpacaEval, because iCliniq includes many domain-specific terms with relatively isolated embeddings.

Solution. To solve Equation 5, we need to express the input deviation Δ in terms of the perturbation δ . Given $\hat{\mathbf{z}} = F_C^{-1}(\mathbf{h}_{Q_1} + \delta)$ and $\mathbf{z} = F_C^{-1}(\mathbf{h}_{Q_1})$, the path integral [1] yields:

$$\|\Delta\|_p = \|\hat{\mathbf{z}} - \mathbf{z}\|_p = \left\| \int_0^1 (\hat{\mathbf{z}} - \mathbf{z}) \nabla F_C^{-1}(\mathbf{z} + t(\hat{\mathbf{z}} - \mathbf{z})) dt \right\|_p.$$

In practice, this integral is approximated numerically. A finer discretization of $t \in [0, 1]$ yields a more accurate approximation but increases computational cost. To keep the defense efficient, we adopt a two-point trapezoidal approximation⁷:

$$\|\Delta\|_p = \|(\hat{\mathbf{z}} - \mathbf{z}) (\nabla F_C^{-1}(\mathbf{z}) + \nabla F_C^{-1}(\hat{\mathbf{z}})) / 2\|_p. \quad (6)$$

Empirically, we find that this two-step approximation works well. Further increasing the number of steps did not lead to a significant improvement in the performance of *PriPert* (Section 6.4). We adopt this approximation as the default for the remainder of this paper.

The solution to this problem is a perturbation that aligns with the most sensitive directions of the inverse mapping, which, as we defined in Section 5, correspond to the least sensitive directions of the forward pass. The specific form of the solution for δ depends on the choice of norms p and q . We derive analytical solutions for common values of p and q :

⁵The local smoothness of neural network mappings states that smaller perturbations are less impactful on model outputs [1].

⁶For cosine similarity, one can map the embeddings onto a unit sphere and consider angular distance. Then, a similar guarantee can be derived by replacing q -norm with angular distance.

⁷For a detailed computational procedure, see [10].

- $p = q = 2$ (L_2 -norm with L_2 constraint): The optimal perturbation δ is the right singular vector of the inverse Jacobian corresponding to its largest singular value, scaled by the budget μ . This can be expressed as $\delta = \mu \cdot \mathbf{v}_{\max}$, where $\mathbf{v}_{\max}$ is the right singular vector of $(\nabla F_C^{-1}(\mathbf{z}) + \nabla F_C^{-1}(\hat{\mathbf{z}}))$ corresponding to its largest singular value.
- $p = 2, q = 0$ (L_2 -norm with L_0 constraint): The optimal solution is to identify the top- $k\%$ elements in the vector $\delta (\nabla F_C^{-1}(\mathbf{z}) + \nabla F_C^{-1}(\hat{\mathbf{z}}))$ based on their magnitude rankings, and subsequently retain only the corresponding elements in $\mathbf{h}_{Q_1}$ while zeroing out all remaining components. Here, $(1 - k)\%$ represents the sparsity ratio that governs the degree of pruning applied to the intermediate representations. We here only present the final perturbed intermediate activation rather than δ itself for brevity.

In our initial empirical evaluations, the solution with the L_0 constraint consistently outperforms the solution with the L_2 constraint⁸. We hypothesize that this performance gap arises because small L_2 -bounded perturbations tend to jitter magnitudes while preserving signs. Such sign information can itself be exploited by adversaries to infer semantic properties of the original input [38]. Unless otherwise specified, all subsequent evaluations therefore adopt the solution with L_0 , which has the additional advantage of lowering the communication cost between client and server due to its sparse nature.

6.3 Evaluation Setup

Attack. We evaluate *PriPert* against *ActInv* using the same attack hyperparameters as in Section 4. Note that it is standard in security to consider adaptive attackers. A natural adaptive strategy here would be to ignore zero entries when matching activations, so as not to be misled by sparsity. Since *ActInv* uses cosine distance, which is inherently insensitive to zero-valued dimensions, it already behaves as such an adaptive attack. This is also why we chose cosine distance over L_2 distance in our attack design.

Metrics. We evaluate *PriPert* from three primary perspectives: privacy, utility, computational overhead. For privacy metrics, we include Precision, Recall, and ROUGE-L to measure the similarity between the reconstructed query and the original query. For utility assessment, we adopt the LLMs-as-judge [41], a state-of-the-art approach for evaluating the quality of LLMs' responses. Specifically, we generate responses to user queries under varying defense methods and intensities, then submit query-response pairs to GPT-4o for comprehensive quality assessment, including correctness, completeness, relevance, clarity, and style. The judge model prompt is included in Figure 6.

Datasets and models. We conduct evaluations on AlpacaEval and iCliniq. We employ Qwen3-0.6B, Falcon3-1B, and Qwen3-30B.

Hyperparameters. The sparsity ratios of *PriPert* are over $\{0.1, 0.3, 0.5, 0.7, 0.9\}$.

6.4 Evaluation Results

Privacy protection. Table 6 and Table 7 show the performance of *PriPert* against *ActInv* on both AlpacaEval and iCliniq. *PriPert* consistently and substantially reduces reconstruction quality, with stronger sparsity (higher ratio) giving stronger protection. For example, on Qwen3-0.6B with AlpacaEval at sparsity 0.7, *ActInv*'s

⁸The scores of ROUGE-L differ by approximately 0.05 to 0.1 when using Qwen3-0.6B and AlpacaEval under similar utility level.

Table 6: The *PriPert*'s performance against *ActInv* in Qwen3-0.6B. Results are presented as mean $\pm$ std.

Dataset	Sparsification Ratio	Precision		Recall		ROUGE-L	
		Qwen3-0.6B	Falcon3-1B	Qwen3-0.6B	Falcon3-1B	Qwen3-0.6B	Falcon3-1B
AlpacaEval	0.1	88.64 $\pm$ 7.05	94.54 $\pm$ 3.22	79.67 $\pm$ 7.24	89.35 $\pm$ 4.77	0.89 $\pm$ 0.05	0.89 $\pm$ 0.07
	0.3	59.88 $\pm$ 10.08	82.89 $\pm$ 5.91	50.70 $\pm$ 7.81	72.59 $\pm$ 6.73	0.68 $\pm$ 0.08	0.70 $\pm$ 0.10
	0.5	37.26 $\pm$ 11.86	58.52 $\pm$ 9.98	30.70 $\pm$ 7.48	45.75 $\pm$ 7.49	0.45 $\pm$ 0.09	0.40 $\pm$ 0.11
	0.7	17.95 $\pm$ 9.37	32.60 $\pm$ 9.55	14.51 $\pm$ 5.57	21.45 $\pm$ 6.03	0.24 $\pm$ 0.08	0.16 $\pm$ 0.07
	0.9	3.45 $\pm$ 4.50	7.06 $\pm$ 7.19	2.44 $\pm$ 2.74	3.55 $\pm$ 3.20	0.05 $\pm$ 0.05	0.02 $\pm$ 0.03
iCliniq	0.1	93.66 $\pm$ 3.72	95.90 $\pm$ 2.98	85.16 $\pm$ 4.35	90.37 $\pm$ 3.27	0.93 $\pm$ 0.04	0.90 $\pm$ 0.04
	0.3	70.76 $\pm$ 8.51	88.62 $\pm$ 4.82	56.51 $\pm$ 5.47	79.22 $\pm$ 4.61	0.70 $\pm$ 0.06	0.80 $\pm$ 0.06
	0.5	49.37 $\pm$ 9.51	70.86 $\pm$ 7.20	36.31 $\pm$ 4.74	57.67 $\pm$ 6.97	0.47 $\pm$ 0.07	0.58 $\pm$ 0.08
	0.7	26.49 $\pm$ 8.29	44.80 $\pm$ 8.35	17.87 $\pm$ 3.94	31.38 $\pm$ 5.82	0.25 $\pm$ 0.05	0.31 $\pm$ 0.07
	0.9	8.22 $\pm$ 5.63	16.41 $\pm$ 7.96	4.39 $\pm$ 2.15	7.86 $\pm$ 3.42	0.07 $\pm$ 0.03	0.06 $\pm$ 0.03

Table 7: The *ActInv*'s effectiveness against different defenses in Qwen3-30B on AlpacaEval and iCliniq. Defense levels correspond to Gaussian noise and activation sparsification as defined in Table 2.

Dataset	M.	L.	Precision				Recall				ROUGE-L			
			Gaussian	Element	Token	Ours	Gaussian	Element	Token	Ours	Gaussian	Element	Token	Ours
AlpacaEval	Qwen3-30B	1	99.39	99.95	99.38	92.67	98.98	98.99	98.62	85.93	0.98	0.98	0.98	0.91
		2	99.34	99.11	99.70	74.03	98.90	98.35	97.88	67.64	0.97	0.96	0.95	0.72
		3	99.73	97.38	98.65	47.80	97.59	95.85	96.44	40.97	0.95	0.94	0.94	0.50
		4	96.92	82.90	86.54	26.34	92.36	76.28	79.52	20.55	0.93	0.86	0.85	0.29
		5	8.65	45.83	46.52	19.45	6.63	37.23	38.60	13.97	0.11	0.65	0.69	0.15
iCliniq	Qwen3-30B	1	99.21	99.94	99.17	95.41	99.58	99.13	98.50	90.40	0.99	0.98	0.99	0.92
		2	99.35	99.46	99.59	75.02	98.81	98.49	98.31	68.58	0.98	0.96	0.97	0.75
		3	99.54	97.61	98.45	54.18	97.98	96.67	97.62	44.08	0.97	0.94	0.95	0.54
		4	98.27	89.25	93.60	30.63	95.99	82.81	86.90	24.34	0.94	0.88	0.89	0.32
		5	8.97	53.68	54.89	21.76	16.10	45.18	46.98	15.62	0.10	0.67	0.72	0.18

Table 8: Utility evaluation of Qwen3-0.6B. Bold marks the best among the three methods at Level 1 (i.e., L.1, weakest defense) within each dataset/metric, while underlining marks the best at Level 5 (i.e., L.5, strongest defense).

Dataset	Metric	GaussianNoise					Sparsification					<i>PriPert</i>				
		L.1	L.2	L.3	L.4	L.5	L.1	L.2	L.3	L.4	L.5	L.1	L.2	L.3	L.4	L.5
AlpacaEval	Accuracy & Correctness	2.05	2.31	2.49	2.10	0.05	2.20	1.63	1.51	0.68	0.07	2.35	2.42	2.33	1.82	<u>1.84</u>
	Completeness & Depth	2.01	1.83	1.86	1.66	0.10	2.01	1.50	1.67	0.14	0.13	2.20	1.67	1.48	1.26	<u>0.96</u>
	Relevance	3.85	3.91	3.69	3.17	0.14	3.72	3.46	1.48	1.46	1.00	3.73	3.64	3.46	3.34	<u>3.14</u>
	Clarity & Organization	3.90	3.78	3.63	3.11	1.16	3.84	3.61	3.58	3.48	3.43	3.89	3.97	3.51	3.48	<u>3.45</u>
	Style & Tone	3.99	3.85	3.65	3.17	1.25	3.88	3.64	3.54	3.46	<u>3.48</u>	3.92	3.85	3.82	3.51	2.27
	Average	3.16	3.14	3.07	2.64	0.54	3.13	2.77	2.36	1.84	1.62	3.22	3.11	2.92	2.63	<u>2.34</u>
iCliniq	Accuracy & Correctness	1.83	1.74	1.48	1.36	0.35	1.63	1.53	1.19	1.13	0.60	1.57	1.53	1.48	1.53	<u>1.32</u>
	Completeness & Depth	1.64	1.69	1.57	1.39	0.28	1.28	1.18	1.10	1.06	<u>0.60</u>	1.18	0.90	0.85	0.76	0.46
	Relevance	2.84	2.65	2.59	2.30	0.97	2.87	2.91	2.09	1.64	0.57	3.11	2.92	2.85	2.52	<u>1.92</u>
	Clarity & Organization	3.47	2.88	2.70	2.88	1.49	2.94	2.86	2.52	1.99	0.54	3.20	3.13	3.03	2.42	<u>2.00</u>
	Style & Tone	3.11	2.66	2.43	2.55	1.52	2.72	2.69	2.53	2.26	0.61	2.80	2.68	2.66	2.34	<u>1.89</u>
	Average	2.58	2.33	2.15	2.10	0.92	2.29	2.23	1.89	1.62	0.58	2.37	2.23	2.17	1.91	<u>1.52</u>

Precision and Recall drop below 30%, compared to over 60% when using traditional sparsification at the same rate (Table 2). ROUGE-L also decreases sharply, indicating that the adversary can no longer recover the semantic content of the prompt. We further observe that, for a fixed defense ratio, Qwen3-0.6B generally provides stronger privacy than Falcon3-1B. For the same sparsity, Qwen3-0.6B generally provides better privacy than Falcon3-1B (e.g., Precision 17.95% vs. 32.60% at sparsity 0.7 on AlpacaEval), while also offering higher

utility, indicating a more favorable privacy–utility trade-off. Finally, we see that iCliniq is generally harder to defend, because domain-specific and highly distinctive medical tokens are easier to reconstruct than generic open-domain vocabulary.

Utility evaluation. Tables 8, 9, and 10 show that *PriPert* preserves utility well at low-to-moderate sparsity levels. Since the responses generated by Qwen3-30B consistently achieve very high scores (above 3.5) in Relevance, Clarity, and Style, we omit these

Table 9: Utility evaluation of Falcon3-1B. Bold marks the best among the three methods at Level 1 (i.e., L.1, weakest defense) within each dataset/metric, while underlining marks the best at Level 5 (i.e., L.5, strongest defense).

Dataset	Metric	GaussianNoise					Sparsification					PriPert				
		L.1	L.2	L.3	L.4	L.5	L.1	L.2	L.3	L.4	L.5	L.1	L.2	L.3	L.4	L.5
AlpacaEval	Accuracy & Correctness	2.09	2.03	1.56	1.55	<u>0.49</u>	2.02	1.38	0.47	0.45	0.58	1.99	1.46	1.00	1.08	0.46
	Completeness & Depth	2.01	2.05	1.53	1.42	<u>0.58</u>	2.00	1.53	0.45	0.50	0.51	1.94	1.48	0.98	0.47	0.52
	Relevance	3.06	3.04	3.02	2.45	<u>0.53</u>	3.45	2.09	0.56	0.50	<u>0.56</u>	3.49	2.43	1.53	0.95	0.48
	Clarity & Organization	3.03	3.02	2.52	2.45	<u>0.55</u>	1.97	1.99	1.46	0.47	0.50	2.46	3.04	1.46	0.96	0.44
	Style & Tone	2.15	2.11	2.12	2.02	<u>0.49</u>	2.05	1.51	1.51	0.50	0.49	2.04	2.98	0.99	0.97	0.49
	Average	2.47	2.45	2.15	1.98	<u>0.53</u>	2.30	1.70	0.89	0.49	0.53	2.38	2.28	1.19	0.89	0.48
iCliniq	Accuracy & Correctness	1.21	1.07	0.94	0.91	<u>0.71</u>	1.46	1.07	1.04	0.80	0.19	1.65	1.50	1.50	0.98	0.56
	Completeness & Depth	2.09	2.15	2.03	2.01	<u>0.75</u>	1.49	1.25	1.00	0.71	0.24	2.02	1.83	1.48	1.06	0.57
	Relevance	2.97	2.48	2.44	2.19	<u>1.24</u>	2.95	2.74	1.52	1.72	0.21	2.94	1.96	1.96	1.06	1.09
	Clarity & Organization	2.45	1.94	1.75	0.96	<u>1.17</u>	2.99	1.96	1.03	0.76	0.16	3.00	2.02	1.54	1.02	0.97
	Style & Tone	2.71	1.31	1.27	1.02	<u>0.97</u>	2.44	1.02	0.75	0.51	0.27	2.48	1.49	0.94	0.96	0.47
	Average	2.28	1.79	1.69	1.42	<u>0.97</u>	2.27	1.61	1.07	0.90	0.22	2.42	1.76	1.48	1.02	0.73

Table 10: Utility evaluation of Qwen3-30B under sparsification and PriPert. Bold marks the best among the three methods at Level 1 (i.e., L.1, weakest defense) within each dataset/metric, while underlining marks the best at Level 5 (i.e., L.5, strongest defense).

Dataset	Metric	Sparsification					PriPert				
		L.1	L.2	L.3	L.4	L.5	L.1	L.2	L.3	L.4	L.5
AlpacaEval	Accuracy	3.67	2.89	2.00	1.74	1.42	3.85	3.46	2.82	2.56	<u>2.29</u>
	Completeness	3.35	2.57	1.91	1.81	1.52	3.64	3.19	2.64	2.36	<u>2.06</u>
	Average	3.51	2.73	1.95	1.78	1.47	3.75	3.33	2.73	2.46	<u>2.17</u>
iCliniq	Accuracy	2.43	2.82	1.86	1.61	1.30	3.34	2.87	2.68	2.38	<u>2.06</u>
	Completeness	2.72	2.23	1.77	1.62	1.33	3.18	2.67	2.49	2.17	<u>1.92</u>
	Average	2.57	2.53	1.81	1.61	1.32	3.26	2.77	2.58	2.28	<u>1.99</u>

Judge Model Prompt

Task. You are a meticulous reviewer. You will receive a *question-answer* pair. Carefully read the pair and evaluate the quality of the answer with respect to the question.

For each of the five criteria below, assign an integer score from 0 (poor) to 4 (excellent):

- (1) **Accuracy & Correctness:** Are the facts accurate? Are any key claims unsupported or incorrect?
- (2) **Completeness & Depth:** Does the answer fully address all aspects of the question? Does it provide meaningful detail and insight?
- (3) **Relevance:** How directly does the answer respond to the question without digression or omission?
- (4) **Clarity & Organization:** Is the explanation clear, logically structured, and free of ambiguity?
- (5) **Style & Tone:** Is the language fluent, professional, and appropriate for the intended audience?

Scoring Scale (per dimension):

Score	Descriptor
4	Excellent (Exceeds expectations)
3	Good (Meets all requirements)
2	Fair (Adequate with minor flaws)
1	Poor (Significant deficiencies)
0	Unacceptable (Fundamentally flawed)

Figure 6: Evaluation rubric used by the judge model.**Table 11: Effectiveness of PriPert against prior prompt inversion attacks on Qwen3-8B and AlpacaEval. We report the arithmetic mean of token-level Precision and Recall (%). Lower is better from the defender’s perspective.**

Attack	Sparsification	PriPert
A1 [17]	18.57	1.91
TBS [7]	81.97	44.92
PIA [25]	95.57	46.11

Table 12: Client-side processing time (s/1k tokens) for defenses.

Defense	GaussianNoise	Sparsification	PriPert
Avg. Time	1.0303 $\pm$ 0.1755	0.9984 $\pm$ 0.1539	4.3703 $\pm$ 0.2987

Table 13: Impact of approximation steps on PriPert performance. We use a sparsification ratio of 0.5.

Approx. Steps	ROUGE-L	Utility	Time (s/1k tokens)
1	0.52	2.30	3.3228
2	0.45	2.36	4.3703
3	0.43	2.36	5.3510

dimensions to avoid confusion and report only the Accuracy and Completeness scores. On Qwen3-0.6B, for instance, PriPert at Level 1 and Level 2 (roughly corresponding to sparsity ≤ 0.5) achieves average LLMs-as-judge scores of 3.53 and 3.16 on AlpacaEval, outperforming traditional sparsification at the same budgets. Moreover,

as expected, all defenses eventually collapse utility at the highest perturbation strengths, indicating that extremely aggressive perturbations are fundamentally incompatible with maintaining model usefulness. A per-criterion view reveals an asymmetry in how utility degrades: Accuracy&Completeness is more sensitive to perturbations than formal aspects like Clarity and Style. For example, in AlpacaEval, with *PriPert* at Level 3 on Qwen3-0.6B, the Accuracy and Completeness scores are around 2.33 and 1.48, while Clarity and Style remain high, at approximately 3.51 and 3.82, respectively. Dataset and architectural differences also play a role. For a fixed defense ratio, Qwen3-0.6B tends to deliver better utility scores than Falcon3-1B. Medical queries from iCliniq also show earlier degradation in Relevance than open-domain AlpacaEval queries at the same nominal budget. This is likely because the specialized terminology in iCliniq concentrates meaning in fewer high-impact tokens, which, when perturbed, can disproportionately distort the perceived correctness and relevance. In summary, sparsity ratios around 0.5 are favorable, providing considerable privacy gains while keeping average utility in an acceptable range.

Case Study. Figure 1 illustrates *PriPert*’s behavior qualitatively. Under a moderate defense setting (level 3; sparsity ratio 0.5), *PriPert* enables the reconstructed prompt to diverge substantially from the original prompt. While a few tokens overlap, the recovered prompt fails to capture the core intent and exhibits pronounced semantic drift. Moreover, the model’s response under the same perturbation remains fluent, coherent, and on-topic. This example demonstrates that *PriPert* can substantially reduce semantic leakage from activations while preserving downstream usefulness.

Effectiveness against prior attacks. We also evaluate *PriPert* against A1 [17], TBS [7], and PIA [25]. We use Qwen3-8B, AlpacaEval, and $Q_1 = 5$, and fix the sparsity ratio to 0.5. For each attack, we report the arithmetic mean of token-level Precision and Recall. Table 11 shows that *PriPert* consistently reduces the effectiveness of all evaluated attacks, often by a large margin. Compared with naive sparsification, *PriPert* yields substantially stronger protection across all baselines. For instance, against A1, the reconstruction score drops from 18.57 under standard sparsification to 1.91 under *PriPert*. Similarly, against TBS and PIA, the scores decrease from 81.97 to 44.92 and from 95.57 to 46.11, respectively.

Defense overhead. To quantify the computational cost of *PriPert*, we measure client-side processing time per 1k tokens over 1,000 trials on an NVIDIA RTX 4090. The average processing times (in seconds) and their corresponding standard deviations are presented in Table 12. Both GaussianNoise and Sparsification incur a relatively low overhead, with *PriPert* having a higher average processing time. This is primarily due to the additional gradient computations involved in *PriPert*. In many privacy-sensitive scenarios (e.g., healthcare, legal advice), this overhead may be acceptable given the strong privacy gains. For latency-critical settings, a natural optimization is to focus on semantic-critical tokens, such as names or rare domain-specific entities, rather than processing all tokens. Please refer to Appendix D for more discussion where the time cost of *PriPert* can be reduced to approximately 1.16 seconds.

Ablation on approximation granularity. Table 13 examines how the number of approximation steps in the path integral affects privacy, utility, and overhead. We evaluate privacy, utility (LLMs-as-judge average score), and client-side processing cost (seconds per 1k

tokens, measured on the same hardware as Table 12). Moving from 1 to 2 steps substantially improves privacy with negligible impact on utility and a moderate increase in cost. Adding a third step yields only marginal additional privacy gains but incurs further overhead. These numbers validate that additional quadrature points improve the approximation, but with rapidly diminishing returns once the dominant directions are captured.

7 Conclusion and Future Work

In this work, we conducted the first comprehensive investigation into prompt leakage within the split inference paradigm for LLMs. We developed *ActInv*, an inversion attack that an honest-but-curious server can use to reconstruct a client’s sensitive input with remarkably high fidelity. This attack proved resilient even against common defenses like activation sparsification and random noise injection, revealing that these naive countermeasures are largely ineffective. We also developed PAF to understand this vulnerability and revealed that privacy vulnerability varies significantly across the model architecture, with layers like activation functions being highly susceptible to leakage. Based on these insights, we designed and evaluated *PriPert*, which injects adversarially calibrated noise into intermediate activations to maximize reconstruction error while preserving model utility with acceptable overhead. The extensive evaluations demonstrated that *PriPert* significantly outperforms traditional defenses, offering a more favorable privacy-utility trade-off.

As a path forward, we believe two promising avenues warrant further investigation. One direction is to design privacy-aware compression techniques that can embed defense properties directly into the model’s structure. Additionally, future work could explore the possibility of designing privacy-sensitive activation functions that maintain model non-linearity while offering greater resistance to inversion attacks.

References

- [1] Stephen P Boyd and Lieven Vandenbergh. 2004. *Convex optimization*. Cambridge university press.
- [2] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, et al. 2023. Quantifying Memorization Across Neural Language Models. In *ICLR*.
- [3] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, et al. 2021. Extracting Training Data from Large Language Models. In *USENIX Security*.
- [4] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, et al. 2024. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* 15, 3 (2024), 39:1–39:45.
- [5] Guanzhong Chen, Zhenghan Qin, Mingxin Yang, Yajie Zhou, Tao Fan, Tianyu Du, and Zenglin Xu. 2024. Unveiling the Vulnerability of Private Fine-Tuning in Split-Based Frameworks for Large Language Models: A Bidirectionally Enhanced Attack. In *CCS*.
- [6] Yuxuan Chen, Rongpeng Li, Xiaoxue Yu, Zhifeng Zhao, and Honggang Zhang. 2024. Adaptive Layer Splitting for Wireless LLM Inference in Edge Computing: A Model-Based Reinforcement Learning Approach. *CoRR* abs/2406.02616 (2024).
- [7] Tian Dong, Yan Meng, Shaofeng Li, Guoxing Chen, Zhen Liu, and Haojin Zhu. 2025. Depth Gives a False Sense of Privacy: LLM Internal States Inversion. In *34th USENIX Security Symposium*.
- [8] In Gim, Caihua Li, and Lin Zhong. 2024. Confidential Prompting: Protecting User Prompts from Cloud LLM Providers. *CoRR* abs/2409.19134 (2024).
- [9] Zecheng He, Tianwei Zhang, and Ruby B. Lee. 2019. Model inversion attacks against collaborative inference. In *ACSAC*.
- [10] Francis Begnaud Hildebrand. 1987. *Introduction to numerical analysis*. Courier Corporation.
- [11] Hongpeng Jin and Yanzhao Wu. 2025. CE-CoLLM: Efficient and Adaptive Large Language Models Through Cloud-Edge Collaboration. In *IEEE International Conference on Web Services*.
- [12] Eric Lehman, Sarthak Jain, Karl Pichotta, Yoav Goldberg, et al. 2021. Does BERT Pretrained on Clinical Notes Reveal Sensitive Data?. In *NAACL*.

- [13] Shiyang Li, Jianshu Chen, Yelong Shen, Zhiyu Chen, et al. 2022. Explanations from Large Language Models Make Small Reasoners Better. *CoRR* (2022).
- [14] Yupei Liu, Yuqi Jia, Jinyuan Jia, and Neil Zhenqiang Gong. 2025. Evaluating LLM-based Personal Information Extraction and Countermeasures. In *Usenix Security* 2025.
- [15] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, et al. 2024. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. In *Findings of the Association for Computational Linguistics*.
- [16] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, et al. 2023. Analyzing Leakage of Personally Identifiable Information in Language Models. In *S&P*.
- [17] Xinjian Luo, Ting Yu, and Xiaokui Xiao. 2025. Prompt Inference Attack on Distributed Large Language Model Inference Frameworks. In *CCS*. ACM, 1739–1753.
- [18] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. LLM-Pruner: On the Structural Pruning of Large Language Models. In *NeurIPS*.
- [19] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. 2024. Split-and-Denoise: Protect large language model inference with local differential privacy. In *ICML*.
- [20] John X. Morris, Wenting Zhao, Justin T. Chiu, Vitaly Shmatikov, et al. 2024. Language Model Inversion. In *ICLR*.
- [21] Giorgos Nikolaou, Tommaso Mencattini, Donato Crisostomi, Andrea Santilli, Yannis Panagakis, and Emanuele Rodolà. 2025. Language Models are Injective and Hence Invertible. *CoRR* abs/2510.15511 (2025).
- [22] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). <https://doi.org/10.48550/arXiv.2303.08774>
- [23] Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. 2021. Unleashing the Tiger: Inference Attacks on Split Learning. In *CCS*.
- [24] Maarten G. Poirot, Praneeth Vepakomma, Ken Chang, Jayashree Kalpathy-Cramer, et al. 2019. Split Learning for collaborative deep learning in healthcare. *CoRR* abs/1912.12115 (2019).
- [25] Wenjie Qu, Yuguang Zhou, Yongji Wu, Tingsong Xiao, Binhang Yuan, Yiming Li, and Jiaheng Zhang. 2025. Prompt Inversion Attack Against Collaborative Inference of Large Language Models. In *IEEE Symposium on Security and Privacy*.
- [26] Siladitya Ray. 2023. A Growing List Of Companies Cracking Down On Use Of ChatGPT By Staffers—Here’s Why.
- [27] Liangqin Ren, Zeyan Liu, Fengjun Li, Kaitai Liang, et al. 2024. PrivDNN: A Secure Multi-Party Computation Framework for Deep Learning using Partial DNN Encryption. *PoPETs* (2024).
- [28] Chris Renzo, Liv Aliberti, Justin Miles, and Joe Kovba. 2024. Large language model inference over confidential data using AWS Nitro Enclaves.
- [29] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, et al. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *ICLR*.
- [30] Robin Staab, Mark Vero, Mislav Balunovic, and Martin T. Vechev. 2024. Beyond Memorization: Violating Privacy via Inference with Large Language Models. In *ICLR*.
- [31] Xuchen Suo. 2024. Signed-Prompt: A New Approach to Prevent Prompt Injection Attacks Against LLM-Integrated Applications. *CoRR* abs/2401.07612 (2024).
- [32] Llama 3 Team. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024).
- [33] Qwen3 Team. 2025. Qwen3 Technical Report. *CoRR* abs/2505.09388 (2025).
- [34] SmoLM2 Team. 2025. SmoLM2: When Smol Goes Big - Data-Centric Training of a Small Language Model. *CoRR* abs/2502.02737 (2025).
- [35] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2017. Distributed Deep Neural Networks Over the Cloud, the Edge and End Devices. In *ICDCS*. IEEE Computer Society, 328–339.
- [36] Dixi Yao, Liyao Xiang, Hengyuan Xu, Hangyu Ye, et al. 2022. Privacy-Preserving Split Learning via Patch Shuffling over Transformers. In *ICDM*.
- [37] Weichen Yu, Tianyu Pang, Qian Liu, Chao Du, et al. 2023. Bag of Tricks for Training Data Extraction from Language Models. In *ICML*, Vol. 202.
- [38] Kai Yue, Richeng Jin, Chau-Wai Wong, Dror Baron, et al. 2023. Gradient Obfuscation Gives a False Sense of Security in Federated Learning. In *USENIX Security*. USENIX Association, 6381–6398.
- [39] Zhixin Zhang, Jiaxin Wen, and Minlie Huang. 2023. ETHICIST: Targeted Training Data Extraction Through Loss Smoothed Soft Prompting and Calibrated Confidence Estimation. In *ACL*.
- [40] Zishuai Zhang, Hainan Zhang, Jiaying Zheng, Ziwei Wang, Yongxin Tong, Jin Dong, and Zhiming Zheng. 2025. A Federated Splitting Framework for LLMs: Security, Efficiency, and Adaptability. *CoRR* abs/2505.15683 (2025).
- [41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS*.

A Proof of Theorem 1

PROOF. We can re-express the $\delta = (\hat{z} - z)J$ by defining $\Delta = \hat{z} - z$, which gives us $\delta = \Delta J$. Finding a solution for Δ can be reformulated as the following optimization problem: $\min_{\Delta} \|\delta - \Delta J\|_2^2$.

To find the optimal solution, we take the gradient of the objective function with respect to Δ and set it to zero:

$$\frac{\partial \|\delta - \Delta J\|_2^2}{\partial \Delta} = -2(\delta - \Delta J)J^T = 0.$$

Simplifying the above expression yields: $\Delta J J^T = \delta J^T$. The solvability of this normal equation depends on the invertibility of $J J^T$, specifically whether it is of full rank. Let us now consider the different cases based on the rank of $J J^T$.

Case 1: $J J^T$ is of full rank. If $J J^T$ is of full rank, its inverse $(J J^T)^{-1}$ exists. Consequently, Δ can be directly solved as: $\Delta = \delta J^T (J J^T)^{-1}$.

Let $\{\sigma_i\}$ and $\{\mu_i\}$ denote eigenvalues and corresponding eigenvectors of $J J^T$ (μ_i is row vector to conform to this paper’s notation convention). Then $(J J^T)^{-1}$ admits the spectral decomposition as follows: $(J J^T)^{-1} = \sum_i \frac{1}{\sigma_i} \mu_i^T \mu_i$. Substituting this into the error expression gives:

$$\begin{aligned} \|\Delta\|_2^2 &= \left\| \delta J^T \left(\sum_i \frac{\mu_i^T \mu_i}{\sigma_i} \right) \right\|_2^2 \\ &= \sum_i \left(\frac{\delta J^T \mu_i^T}{\sqrt{\sigma_i}} \right)^2 = \sum_i \left(\frac{\|\delta\|_2 \|J^T\|_2 \cos \theta_i}{\sqrt{\sigma_i}} \right)^2, \end{aligned}$$

where we use orthogonality of eigenvectors μ_i , and θ_i denotes the angle between X and μ_i .

Case 2: $J J^T$ is not of full rank. When $J J^T$ is not of full rank, its inverse does not exist. To this end, we employ regularization by adding an identity matrix I to $J J^T$ in the normal equation as follows:

$$\Delta(J J^T + I) = \delta J^T.$$

The solution for this regularized normal equation is then given by:

$$\Delta = \delta J^T (J J^T + I)^{-1}.$$

$(J J^T + I)$ and $J J^T$ share the same eigenvectors, $\{\mu_i\}$, but their eigenvalues are shifted by 1. The eigenvalues of $(J J^T + I)$ are $\{\sigma_i + 1\}$. Applying the same spectral decomposition logic as in Case 1, we derive:

$$\begin{aligned} \|\Delta\|_2^2 &= \left\| \delta J^T \left(\sum_i \frac{\mu_i^T \mu_i}{\sigma_i + 1} \right) \right\|_2^2 \\ &= \sum_i \left(\frac{\delta J^T \mu_i^T}{\sqrt{\sigma_i + 1}} \right)^2 = \sum_i \left(\frac{\|\delta J^T\|_2 \cos \theta_i}{\sqrt{\sigma_i + 1}} \right)^2. \end{aligned} \quad (7)$$

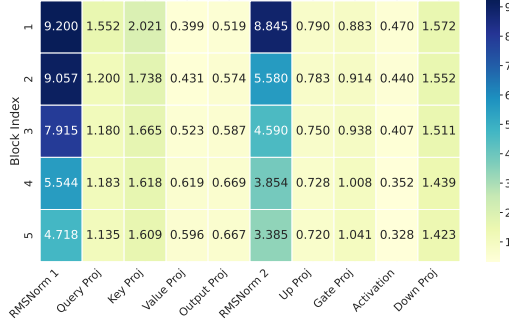
Final expression. The regularized solution in Case 2 guarantees a larger upper bound on $\|\Delta\|_2^2$ compared to the solution in Case 1, as the denominator $\sqrt{\sigma_i + 1}$ becomes smaller than $\sqrt{\sigma_i}$. To unify both cases, we adopt the regularized form as a conservative estimate, leading to the final expression:

$$\|\Delta\|_2^2 \leq \sum_i \left(\frac{\|\delta J^T\|_2 \cos \theta_i}{\sqrt{\sigma_i + 1}} \right)^2. \quad (8)$$

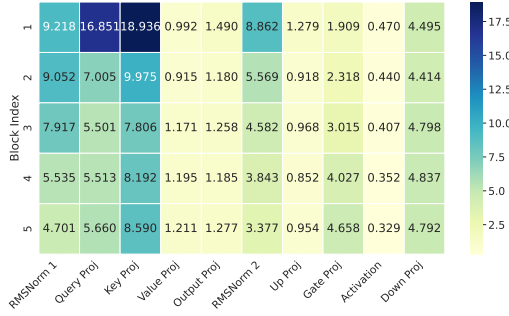
□

B PAF Evaluation in Llama-3.2-1B and SmoLM2-1.7B

Figures 7 and 8 present PAF values for different layers of Llama-3.2-1B and SmoLM2-1.7B, respectively.



(a) Expected PAF



(b) Max-PAF

Figure 7: Comparison of different layers' sensitivity in Llama-3.2-1B.

C Proof of Theorem 2

PROOF. We know $\delta = F_C(\hat{z}) - F_C(z) = \int_0^1 \nabla F_C(z + t\Delta) \Delta dt$. Taking the q -norm and using the consistency of induced norms,

$$\|\delta\|_q \leq \int_0^1 \|\nabla F_C(z + t\Delta)\|_q dt \cdot \|\Delta\|_q \leq C\|\Delta\|_q.$$

Since $\|\delta\|_q \leq \mu$, we obtain $\mu \leq C\|\Delta\|_q$, i.e., $\|\Delta\|_q \geq \mu/C$. This proves the first part.

For the second part, the first part gives $\|\Delta\|_q \geq \mu/C > d_{\min}/2$. Let $e_{\min}$ be the token achieving $d_{\min}$. By the triangle inequality,

$$\|\hat{z} - e_{\min}\|_q \geq \|\|z - e_{\min}\|_q - \|\hat{z} - z\|_q\| = d_{\min} - \|\Delta\|_q,$$

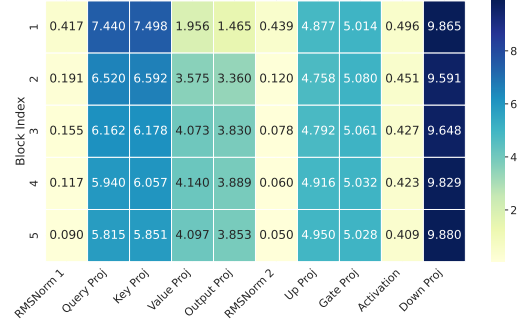
provided $\|\Delta\|_q \leq d_{\min}$. If $\|\Delta\|_q > d_{\min}$, then trivially $\hat{z}$ is farther from z than $e_{\min}$, so the conclusion holds. Otherwise ($\|\Delta\|_q \leq d_{\min}$), we have $d_{\min} - \|\Delta\|_q < \|\Delta\|_q$ because $\|\Delta\|_q > d_{\min}/2$. Thus

$$\|\hat{z} - e_{\min}\|_q \leq d_{\min} - \|\Delta\|_q < \|\Delta\|_q = \|\hat{z} - z\|_q.$$

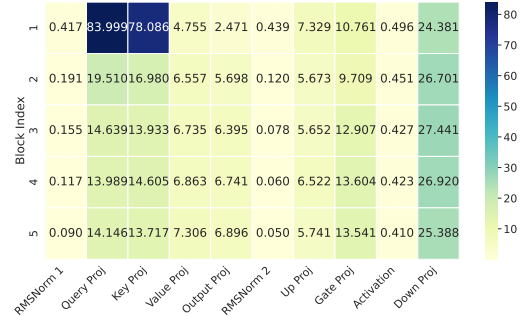
Hence $\hat{z}$ is closer to $e_{\min}$ than to z , so the true token is not the nearest neighbor. $\square$

D Overhead Optimization

To mitigate the computational overhead of *PriPert*, we can apply the gradient-based optimization exclusively to a subset of protected tokens, such as privacy-sensitive entities or domain-specific terms. Let $r \in [0, 1]$ denote the protected ratio, representing the proportion of tokens subject to *PriPert* optimization. Let T represent the time required for a standard forward pass. Since the overhead of baseline



(a) Expected PAF



(b) Max-PAF

Figure 8: Comparison of different layers' sensitivity in SmolLM2-1.7B.

Table 14: Processing overhead for domain-specific terms and privacy-sensitive terms on iCliniq (per 1k tokens).

	Domain-Specific	Privacy-Sensitive
Ratio (r)	19.22% $\pm$ 6.69%	4.22% $\pm$ 4.13%
Expected Time	1.58T	1.13T
Actual Time	1.64s $\pm$ 0.28s	1.16s $\pm$ 0.21s

defenses (e.g., Gaussian noise or sparsification) is negligible, their total processing time remains approximately T . The expected execution time for *PriPert* is modeled as $T + 3rT$, where the term $3rT$ accounts for the additional gradient computation and backpropagation costs associated with the optimized tokens.

Table 14 reports the processing overhead analysis on iCliniq. We employ GPT-5 to identify domain-specific and privacy-sensitive tokens within user queries and calculate their average occurrence ratios (r). We compare the theoretical expected time against the actual measured execution time, utilizing the same experimental setup described in Table 12. The results demonstrate that, since domain-specific and privacy-sensitive terms constitute only a small fraction of the input, the actual computational overhead of *PriPert* remains low in practice.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009